

ENGS 177: Decision Making Under Uncertainty

Homework 3

Name: Taka Khoo • **Instructor:** Professor Wesley Marrero

Due: May 15, 2026

Note: Claude (Anthropic) was used to sanity-check reasoning, verify arithmetic, and help polish the L^AT_EX formatting.

Problem 1: Inventory Management via Finite-Horizon DP [30 pts]

A company needs to manage its inventory over a finite planning horizon of 5 days. Each day, the company can either order new stock or use existing stock to meet demand. There is a holding cost of $h = 1$ per unit per day and an ordering cost of $c = 3$ per unit. The objective is to minimize the total cost over the planning horizon. The initial stock is 5 units. Suppose the expected demand for the next five days is $d_t = [2, 3, 1, 4, 2] \forall t$. Code the appropriate dynamic programming algorithm to obtain the optimal cost and policy.

Formulation

We model this as a deterministic finite-horizon dynamic program (Lecture 6, backward induction).

State. $s_t \in \{0, 1, \dots, S_{\max}\}$: inventory at the *start* of day t , before ordering. We set $S_{\max} = 12$ (total demand).

Action. $a_t \geq 0$: units ordered on day t (integer), subject to feasibility $s_t + a_t \geq d_t$.

Transition. $s_{t+1} = s_t + a_t - d_t$.

Stage cost. Ordering plus holding on ending inventory:

$$g_t(s_t, a_t) = c \cdot a_t + h \cdot (s_t + a_t - d_t) = 3a_t + (s_t + a_t - d_t).$$

Terminal condition. $V_6(s) = 0$ for all $s \geq 0$.

Bellman equation (backward induction). For $t = 5, 4, \dots, 1$:

$$V_t(s) = \min_{a \geq \max(0, d_t - s)} \left\{ g_t(s, a) + V_{t+1}(s + a - d_t) \right\}.$$

Python implementation

```

1  import numpy as np
2
3  T, h, c, s0 = 5, 1, 3, 5
4  d = [2, 3, 1, 4, 2]
5  max_inv = 12
6
7  V = np.full((T + 2, max_inv + 1), np.inf)
8  policy = np.full((T + 1, max_inv + 1), -1, dtype=int)
9  V[T + 1, :] = 0 # terminal condition
10
11 for t in range(T, 0, -1): # backward induction
12     dt = d[t - 1]
13     for s in range(max_inv + 1):
14         best_cost, best_a = np.inf, 0
15         for a in range(max(0, dt - s), max_inv - s + 1):
16             s_next = s + a - dt
17             if s_next < 0 or s_next > max_inv:
18                 continue
19             cost = c * a + h * s_next + V[t + 1, s_next]
20             if cost < best_cost:
21                 best_cost, best_a = cost, a
22         V[t, s] = best_cost
23         policy[t, s] = best_a

```

Value function table

$V_t(s)$	$s = 0$	$s = 1$	$s = 2$	$s = 3$	$s = 4$	$s = 5$	$s = 6$	$s = 7$
$t = 6$	0	0	0	0	0	0	0	0
$t = 5$ ($d_5=2$)	6	3	0	1	2	3	4	5
$t = 4$ ($d_4=4$)	18	15	12	9	6	4	2	4
$t = 3$ ($d_3=1$)	21	18	16	14	12	10	9	8
$t = 2$ ($d_2=3$)	30	27	24	21	19	18	17	16
$t = 1$ ($d_1=2$)	36	33	30	28	26	24	23	23

Optimal cost and policy

Starting from $s_1 = 5$, the minimum total cost is $V_1(5) = 24$.

The optimal policy (forward trace):

Day t	1	2	3	4	5
Demand d_t	2	3	1	4	2
Start inventory s_t	5	3	0	0	0
Order a_t^*	0	0	1	4	2
End inventory	3	0	0	0	0
Day cost	3	0	3	12	6

Interpretation. The optimal strategy uses the initial surplus to cover days 1–2 without ordering (incurring only a holding cost of \$3 on day 1). From day 3 onward the stock is depleted, so the company orders exactly the day’s demand, minimizing holding cost to zero. The total cost is $3 + 0 + 3 + 12 + 6 = 24$.

Problem 2: Markov Decision Process

[40 pts]

Consider a discrete-time system that moves between states $\{s_1, s_2, s_3, s_4\}$ according to the transition probability matrix:

$$P = \begin{bmatrix} 0.3 & 0.4 & 0.2 & 0.1 \\ 0.2 & 0.3 & 0.5 & 0 \\ 0.1 & 0 & 0.8 & 0.1 \\ 0.4 & 0 & 0 & 0.6 \end{bmatrix}.$$

At each point in time, you may leave the system and receive a reward of $\gamma = \$20$, or remain in the system and receive a reward $\$i$ if the system occupies state s_i . If you decide to remain in the system, its state at the next decision epoch is determined by P . Assume a discount rate of 0.9.

(a) MDP Formulation

[5 pts]

Formulate this problem as an MDP.

Following the Lecture 5 formulation, the MDP is specified by the tuple (S, A, P, r, λ) :

- **States.** $S = \{s_1, s_2, s_3, s_4, s_0\}$, where s_0 is an absorbing “out” state entered after leaving.
- **Actions.** $A(s_i) = \{\text{stay}, \text{leave}\}$ for $i = 1, \dots, 4$; $A(s_0) = \{\text{stay}\}$.
- **Transition probabilities.**
 - stay in s_i : next state drawn from row i of P .
 - leave from s_i : deterministic transition to s_0 .
 - s_0 : self-loop with probability 1.
- **Rewards.** $r(s_i, \text{stay}) = i$ for $i = 1, \dots, 4$; $r(s_i, \text{leave}) = \gamma = 20$; $r(s_0, \text{stay}) = 0$.

- **Discount factor.** $\lambda = 0.9$.

The **Bellman optimality equation** for the four active states is:

$$v^*(s_i) = \max \left\{ \gamma, \quad i + \lambda \sum_{j=1}^4 P_{ij} v^*(s_j) \right\}, \quad i = 1, \dots, 4.$$

(b) Modified Policy Iteration

[15 pts]

Use modified policy iteration, with $\epsilon = 0.01$ and $m_n = n + 10$ as the sequence of integers in the partial policy evaluation step, to find a stationary policy which maximizes the expected total discounted reward.

We apply the modified policy iteration algorithm from Lecture 7 (slide 24). Initialize $v^0 = \mathbf{0}$, $\epsilon = 0.01$, stopping threshold $\frac{\epsilon(1-\lambda)}{2\lambda} = \frac{0.01 \times 0.1}{1.8} \approx 0.000556$.

Iteration $n = 0$ ($m_0 = 10$):

- *Policy improvement:* $Q(\text{stay}) = r + 0.9 P v^0 = [1, 2, 3, 4]$. Since $\gamma = 20 > Q_i$ for all i , set $\pi_1 = (\text{leave}, \text{leave}, \text{leave}, \text{leave})$.
- $u_0^0 = [20, 20, 20, 20]$, $\|u_0^0 - v^0\|_\infty = 20 > 0.000556$.
- *Partial evaluation* ($m_0 = 10$ steps under π_1): since leaving gives reward 20 and transitions to the absorbing state, every iterate equals $[20, 20, 20, 20]$.
- $v^1 = [20, 20, 20, 20]$.

Iteration $n = 1$ ($m_1 = 11$):

- $Q(\text{stay}) = r + 0.9 P v^1 = [19, 20, 21, 22]$. Since $Q_1 < 20$ and $Q_2 = 20$: $\pi_2 = (\text{leave}, \text{stay}, \text{stay}, \text{stay})$.
- $u_1^0 = [20, 20, 21, 22]$, $\|u_1^0 - v^1\|_\infty = 2$.
- *Partial evaluation* (11 steps under π_2): $v^2 = [20.00, 22.93, 24.83, 24.35]$.

Iteration $n = 2$ ($m_2 = 12$):

- $Q(\text{stay}) = [21.31, 22.96, 24.87, 24.35]$. All exceed $\gamma = 20$: $\pi_3 = (\text{stay}, \text{stay}, \text{stay}, \text{stay})$.
- $\|u_2^0 - v^2\|_\infty = 1.31$.
- *Partial evaluation* (12 steps): $v^3 = [23.33, 24.76, 26.56, 26.79]$.

Subsequent iterations continue with $\pi = (\text{stay}, \text{stay}, \text{stay}, \text{stay})$ and the span norm of $u_n^0 - v^n$ decreases:

n	m_n	$v^n(s_1)$	$v^n(s_2)$	$v^n(s_3)$	$v^n(s_4)$	$\ u_n^0 - v^n\ _\infty$
3	13	23.33	24.76	26.56	26.79	0.0748
4	14	23.91	25.34	27.13	27.37	0.0171
5	15	24.04	25.47	27.27	27.50	0.0035
6	16	24.07	25.50	27.30	27.53	0.0007
7	17	24.08	25.51	27.30	27.54	0.0001

At $n = 7$: $\|u_7^0 - v^7\|_\infty = 0.000109 < 0.000556$. **Stop.**

$$\pi_\epsilon = (\text{stay}, \text{stay}, \text{stay}, \text{stay}), \quad v_\epsilon \approx (24.08, 25.51, 27.30, 27.54).$$

The ϵ -optimal policy is to **stay in every state**.

(c) Policy Iteration

[15 pts]

Verify your solution to part (2b) using policy iteration to find a stationary policy which maximizes the expected total discounted reward.

We apply the exact policy iteration algorithm from Lecture 7.

PI Iteration 0: $\pi_0 = (\text{leave}, \text{leave}, \text{leave}, \text{leave})$.

- *Policy evaluation:* $v^{\pi_0} = [20, 20, 20, 20]$.
- *Improvement:* $Q(\text{stay}) = [19, 20, 21, 22]$. Switch s_2 (tie, keep leave), s_3, s_4 to stay: $\pi_1 = (\text{leave}, \text{stay}, \text{stay}, \text{stay})$.

PI Iteration 1: $\pi_1 = (\text{leave}, \text{stay}, \text{stay}, \text{stay})$.

- *Policy evaluation:* solve $(I - 0.9 P_{\pi_1}) v = r_{\pi_1}$ where $r_{\pi_1} = (20, 2, 3, 4)$ and P_{π_1} sends s_1 to the absorbing state:

$$v^{\pi_1} = (20.00, 23.06, 24.97, 24.35).$$

- *Improvement:* $Q(s_1, \text{stay}) = 1 + 0.9 \cdot [0.3(20) + 0.4(23.06) + 0.2(24.97) + 0.1(24.35)] = 21.39 > 20$. Switch s_1 to stay: $\pi_2 = (\text{stay}, \text{stay}, \text{stay}, \text{stay})$.

PI Iteration 2: $\pi_2 = (\text{stay}, \text{stay}, \text{stay}, \text{stay})$.

- *Policy evaluation:* solve $(I - 0.9 P) v = r$ where $r = (1, 2, 3, 4)$.
The system $(I - 0.9P) v = r$ is:

$$\begin{bmatrix} 0.73 & -0.36 & -0.18 & -0.09 \\ -0.18 & 0.73 & -0.45 & 0 \\ -0.09 & 0 & 0.28 & -0.09 \\ -0.36 & 0 & 0 & 0.46 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}.$$

Solving:

$$v^{\pi_2} = (24.0775, 25.5087, 27.3053, 27.5389).$$

- *Improvement:* $Q(s_i, \text{stay}) = v^{\pi_2}(s_i) \geq 24.08 > 20 = \gamma$ for all i . No state switches.
Converged.

$$\pi^* = (\text{stay}, \text{stay}, \text{stay}, \text{stay}), \quad v^* = (24.0775, 25.5087, 27.3053, 27.5389).$$

This confirms the modified policy iteration result from part (b). Both algorithms converge to the same optimal policy: **always stay**.

(d) Smallest γ for leaving state 2

[5 pts]

What is the smallest value of γ so that it is optimal to leave the system in state 2? [Hint: Rely on your solution approach to (2c).]

We use the policy iteration framework from (2c). Under the all-stay policy, the values $v^*(s_i) = (24.08, 25.51, 27.31, 27.54)$ do not depend on γ , since the “stay” rewards are fixed at $\$i$.

As γ increases, the first state where leaving becomes optimal is s_1 (lowest value, $v^*(s_1) = 24.08$). Once $\gamma > 24.08$, the policy changes to $\pi = (\text{leave}, \text{stay}, \text{stay}, \text{stay})$, which affects the values of s_2, s_3, s_4 because transitions to s_1 now yield γ instead of $v^*(s_1)$.

Under $\pi = (\text{leave } s_1, \text{stay } s_2, s_3, s_4)$, the values of the staying states satisfy:

$$\begin{aligned} v(s_2) &= 2 + 0.9[0.2\gamma + 0.3v(s_2) + 0.5v(s_3)], \\ v(s_3) &= 3 + 0.9[0.1\gamma + 0.8v(s_3) + 0.1v(s_4)], \\ v(s_4) &= 4 + 0.9[0.4\gamma + 0.6v(s_4)]. \end{aligned}$$

Solving for each value as a linear function of γ :

$$\begin{aligned} v(s_4) &= \frac{4 + 0.36\gamma}{0.46} = 8.696 + 0.783\gamma, \\ v(s_3) &= \frac{3 + 0.09\gamma + 0.09v(s_4)}{0.28} = 13.509 + 0.573\gamma, \\ v(s_2) &= \frac{2 + 0.18\gamma + 0.45v(s_3)}{0.73} = 11.067 + 0.600\gamma. \end{aligned}$$

The agent is indifferent between staying and leaving s_2 when $\gamma = v(s_2)$:

$$\gamma = 11.067 + 0.600\gamma \implies 0.400\gamma = 11.067 \implies \gamma^* = \frac{11.067}{0.400} \approx 27.65.$$

Verification. At $\gamma^* \approx 27.65$: $v(s_3) = 29.35 > 27.65$ and $v(s_4) = 30.34 > 27.65$, so s_3 and s_4 still prefer to stay. Also, $Q(s_1, \text{stay}) = 26.44 < 27.65 = \gamma^*$, confirming s_1 still prefers to leave. The policy is consistent.

$$\gamma^* \approx 27.65.$$

Problem 3: ε -Greedy Multi-Armed Bandit [20 pts]

Consider a four-armed bandit whose actions are labeled 1, 2, 3, 4. The agent follows an ε -greedy strategy with sample-average action-value estimates, all initialized to $Q_1(a) = 0$ for every action a .

During the first five interaction steps, the chosen arm and the received reward were recorded as follows:

time step t	1	2	3	4	5
arm a_t	1	2	2	2	3
reward r_t	1	1	2	2	0

With probability ε , the agent explores by picking an arm uniformly at random; otherwise, it exploits by selecting any arm whose current action-value estimate is maximal (breaking ties arbitrarily).

We first track the sample-average Q -values (Lecture 8) at each step:

$$Q_{t+1}(a) = \begin{cases} \frac{\sum_{\tau=1}^t r_{\tau} \mathbf{1}\{a_{\tau} = a\}}{N_t(a)}, & \text{if } N_t(a) > 0, \\ 0, & \text{if } N_t(a) = 0, \end{cases}$$

where $N_t(a) = \sum_{\tau=1}^t \mathbf{1}\{a_{\tau} = a\}$ is the number of times arm a has been pulled through step t .

t	$Q_t(1)$	$Q_t(2)$	$Q_t(3)$	$Q_t(4)$	Greedy arm(s)	a_t chosen
1	0	0	0	0	{1, 2, 3, 4} (all tied)	1
2	1	0	0	0	{1}	2
3	1	1	0	0	{1, 2} (tied)	2
4	1	1.5	0	0	{2}	2
5	1	5/3	0	0	{2}	3

(a) Time steps where exploration *must* have occurred [10 pts]

At which time steps must exploration have occurred?

Exploration *must* have occurred whenever the chosen arm is **not** among the current greedy arms (i.e., the action cannot have resulted from exploitation):

- $t = 2$: The unique greedy arm is $\{1\}$ (with $Q_2(1) = 1 > 0$), but arm 2 was chosen. Exploitation would have selected arm 1, so this **must** be exploration.
- $t = 5$: The unique greedy arm is $\{2\}$ (with $Q_5(2) = 5/3 > 1 > 0$), but arm 3 was chosen. This **must** be exploration.

Exploration must have occurred at $t = 2$ and $t = 5$.

(b) Time steps where exploration *could* have occurred [10 pts]

At which time steps could (but need not) exploration have occurred?

At any time step, the ε -greedy agent explores with probability $\varepsilon > 0$, choosing uniformly at random from all four arms. Even when the chosen arm happens to coincide with a greedy arm, exploration could still have been the mechanism (the random draw just happened to land on the greedy arm). We check the remaining steps:

- $t = 1$: All $Q_1(a) = 0$, so $\{1, 2, 3, 4\}$ are all greedy. Arm 1 was chosen; this could be exploitation (tie broken to arm 1) *or* exploration (random draw landed on arm 1). Exploration is possible but not required.
- $t = 3$: $Q_3(1) = Q_3(2) = 1$, so $\{1, 2\}$ are tied as greedy. Arm 2 was chosen; this could be exploitation (tie broken to arm 2) or exploration. Exploration is possible but not required.
- $t = 4$: $Q_4(2) = 1.5$ is the unique maximum. Arm 2 was chosen; this could be exploitation (greedy selection) or exploration (random draw landed on arm 2 with probability $\varepsilon/4$). Exploration is possible but not required.

Exploration could (but need not) have occurred at $t = 1$, $t = 3$, and $t = 4$.

Problem 4: Term Project Update [10 pts]

Provide a revised formulation of the decision-making problem proposed for your term project and a preliminary solution technique, as applicable.

(a) Progress since Homework 2**[5 pts]**

Has your problem formulation changed since you submitted Homework 2? If so, describe how it has changed. If not, describe your progress (e.g., parameterization of model components, data collection, algorithmic development, proof updates) since the submission of your previous homework assignment.

Our POMDP-based regime-switching asset allocation formulation from Homework 2 **has not changed**. We retain the same state, action, observation, and transition structure. Since HW2, our progress has been concentrated in three areas:

1. **Parameterization.** We calibrated the hidden Markov model (HMM) for regime dynamics using monthly S&P 500 and 10-year Treasury returns from 1990–2024. We fit a 3-regime model via the Baum–Welch (EM) algorithm, yielding estimated regime-transition matrix $\hat{P}_{\text{regime}}$, per-regime return means $\hat{\mu}_k$, and covariances $\hat{\Sigma}_k$ for $k \in \{\text{bull, neutral, bear}\}$.
2. **Algorithmic development.** We implemented the QMDP approximation solver from HW2’s formulation. The underlying fully-observable MDP is solved by value iteration (Lecture 7) on the discretized weight–regime state space. Belief updates use the standard Bayesian filter (Lecture 2) applied to the HMM observation model.
3. **Preliminary results.** A backtest over the 2000–2024 out-of-sample period shows the QMDP policy achieves a Sharpe ratio improvement over the static 60/40 benchmark. The regime filter correctly identifies the 2008 crisis and 2020 drawdown as “bear” regimes, triggering defensive reallocations.

(b) Planned solution strategy**[5 pts]**

What is your planned solution strategy? Is it model-based or model-free? Justify.

Our strategy is **model-based**. Justification:

- **Known model structure.** We have a parametric model of the environment: the HMM defines the regime-transition dynamics $P(s'|s)$, the observation likelihood $P(o|s)$, and the reward function $R(s,a)$. All are estimated from historical data.
- **QMDP as an approximate solver.** The QMDP approximation (Littman et al., 1995) solves the fully-observable MDP via value iteration and weights the state-action values by the belief. This requires the transition model for the Bellman backup and is inherently model-based.
- **Why not model-free?** Model-free methods (e.g., Q-learning, SARSA from Lectures 9–10) learn from online interaction with the environment. In our financial setting, (i) the cost of real-world exploration is prohibitive, (ii) we have decades of historical data sufficient for model calibration, and (iii) the POMDP structure (hidden regimes) maps naturally to Bayesian filtering, which is a model-based operation. A model-free approach would also struggle with the partial observability, since standard temporal-difference methods assume full state access.

The model-based pipeline is: *calibrate HMM $\rightarrow$ solve underlying MDP via value iteration $\rightarrow$ combine with belief filter via QMDP $\rightarrow$ backtest and sensitivity analysis.*